快速开始

English

SANPO SPINE 机器人集成开发板用于将电脑或主控板(树莓派/Jetson等)连接到 CAN、CAN FD 和 RS485 设备/关节电机。可通过 **USB** 和 **SPI** 控制 4 路 独立 CAN/CAN FD 和 4 路 独立 RS485。

本说明适用于 V8 版本的 升级版(STM32G系列芯片)。如果使用 V4 版本的基础版、隔离版(STM32F系列芯片)，请访问 V4 版本使用说明。
除了V4版本的标准功能以外，V8版本已增加 FDCAN，SocketCAN，CANgaroo，透传 RS485串口 等更多功能。

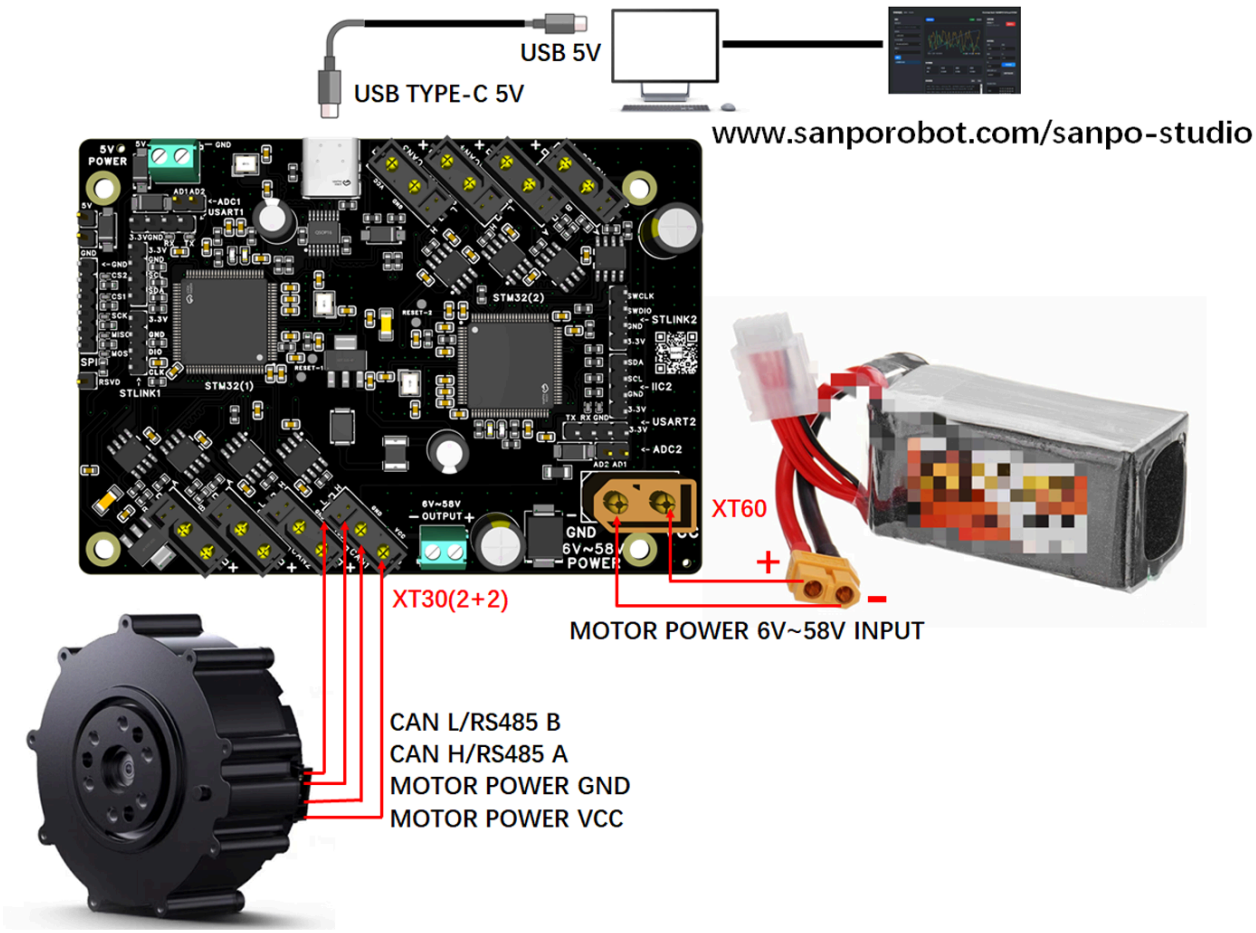
> **提示：** 本说明文档已针对 **AI** 进行过友好化处理，连接好电脑/主控板，电源和电机后，可以把文档和样例程序发给AI，进行控制逻辑的编程。

首次接入和供电

通过 USB 控制 FDCAN/CAN/RS485 设备

1. 硬件接线和供电，XT60接入动力电源给电机供电，XT30(2+2)接入电机CAN/RS485集成接口，USB Type-C 接入电脑或者主控板(Jetson/树莓派等) 给SANPO SPINE集成板供电。

SANPO SPINE
USB to CAN/RS485



2. 点击打开线上调试工具 SANPO Studio https://sanporobot.com/sanpo-studio

- 选择合适的选项 `FDCAN/CAN通用电机测试`，`RS485通用电机测试`，`Cybergear电机测试`，`宇树电机测试`
- 点击 `连接` 按钮

- 弹出的选择框中，选择 `SANPO Studio Management Port`，注意 SANPO Studio 使用 **协议模式**，请选择管理串口，不要选择RS485透传串口(**透传模式**使用)。



- 发送 `FDCAN/CAN/RS485` 数据进行测试
  SANPO Motor，小米CyberGear，宇树GM8010电机可以直接选择右下角的自动化测试，进行正弦低速转动，观察电机平滑度和控制精度



3. 其他控制方式

SANPO SPINE 支持透传和协议模式，适用于各种场景。

> **协议模式：** 发送数据时增加指定的报文头和尾，SANPO SPINE 收到报文，去除头和尾，把原始数据再转发给FDCAN/CAN/RS485设备。
> **透传模式：** 原始数据直接转发给FDCAN/CAN/RS485设备，SANPO SPINE 不做任何处理。

调试工具除了使用协议模式下的 **SANPO Studio**，你依然可以使用其他工具和透传模式，包括宇树电机，小米电机等原厂上位机软件。
以下控制模式适用于所有品牌的关节电机，可根据使用习惯自由选择。

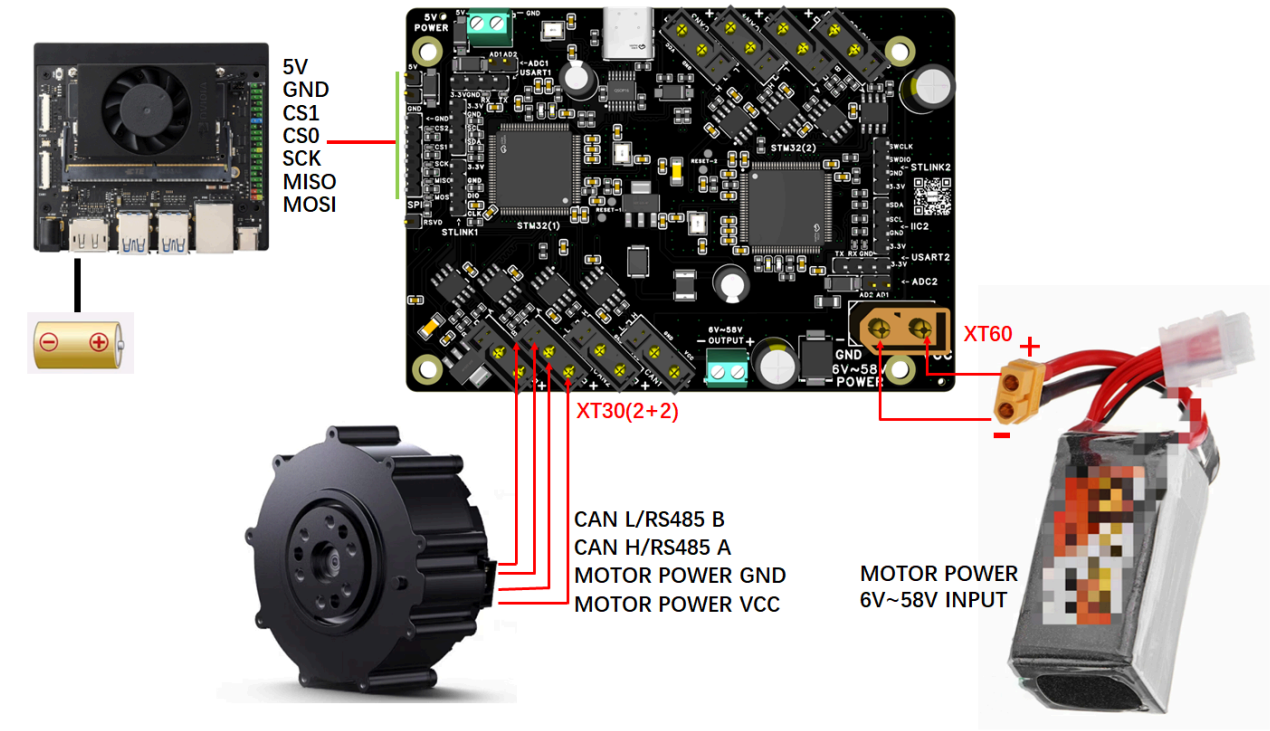
控制模式	文档	调试工具	样例程序	数据接口
USB 转 RS485 透传模式	使用说明	串口助手 宇树原厂上位机(仅宇树电机) 或者其他任意串口工具	RS485 透传样例逐步说明	RS485 透传串口
USB 转 RS485 协议模式	使用说明	SANPO Studio	宇树 GM8010 电机控制逐步说明 SANPO Motor 控制样例	SANPO 管理串口
USB 转 FDCAN/CAN 透传模式	Linux SocketCAN PythonCAN	CANgaroo(Windows) CANgaroo(Linux)	小米 CyberGear SocketCAN 逐步说明 J8108-R8 FDCAN SocketCAN 只读测试	CANX 网络接口(不使用串口)
USB 转 FDCAN 协议模式	使用说明	SANPO Studio	J8108-R8 FDCAN 管理口只读测试	SANPO 管理串口
USB 转 CAN 协议模式	使用说明	SANPO Studio 小米原厂上位机(仅小米电机)	SANPO Motor控制样例	SANPO 管理串口

通过 SPI 控制 FDCAN/CAN/RS485 设备

1. XT60接入动力电源给电机供电，XT30(2+2)接入电机CAN/RS485集成接口，SPI(MOSI/MISO/SCK/CS0/CS1) 排针接入主控板(Jetson/树莓派等)SPI接口，5V 排针接入主控板给 SANPO SPINE 集成板供电。

> **提示：** 除了5V排针，依然可以使用USB给集成板供电，建议保留5V排针中的接地GND连接主控板，可保证 SPI 高速信号稳定。

点击查看 树莓派SPI接口示例，Jetson SPI接口示例



2. SPI 控制使用协议模式，发送数据时增加指定的报文头和报文尾，SANPO SPINE 收到数据后，去掉增加的头和尾，把原始数据再转发给 FDCAN/CAN/RS485 设备。

- 查看 SPI 转 CAN 协议说明和样例程序
- 查看 SPI 转 FDCAN 协议说明和样例程序
- 查看 SPI 转 RS485 协议说明和样例程序

其他功能

- 离线批量执行：保存并自动执行一组电机指令。
- 原厂固件二次开发：基于原厂固件继续开发，保留原厂通信转发功能并增加传感器等外设。
- 自定义固件开发：基于硬件配置框架，完全自定义所有功能。
- 接口和引脚说明：STM32芯片外设配置引脚说明。

- 固件更新：使用 ST-LINK 更新两颗 STM32芯片的软件。

重要： 更新或者自定义 SANPO SPINE 集成板固件时不要执行全片擦除。全片擦除会删除出厂识别信息、已保存的通信参数和离线批次。

常见问题

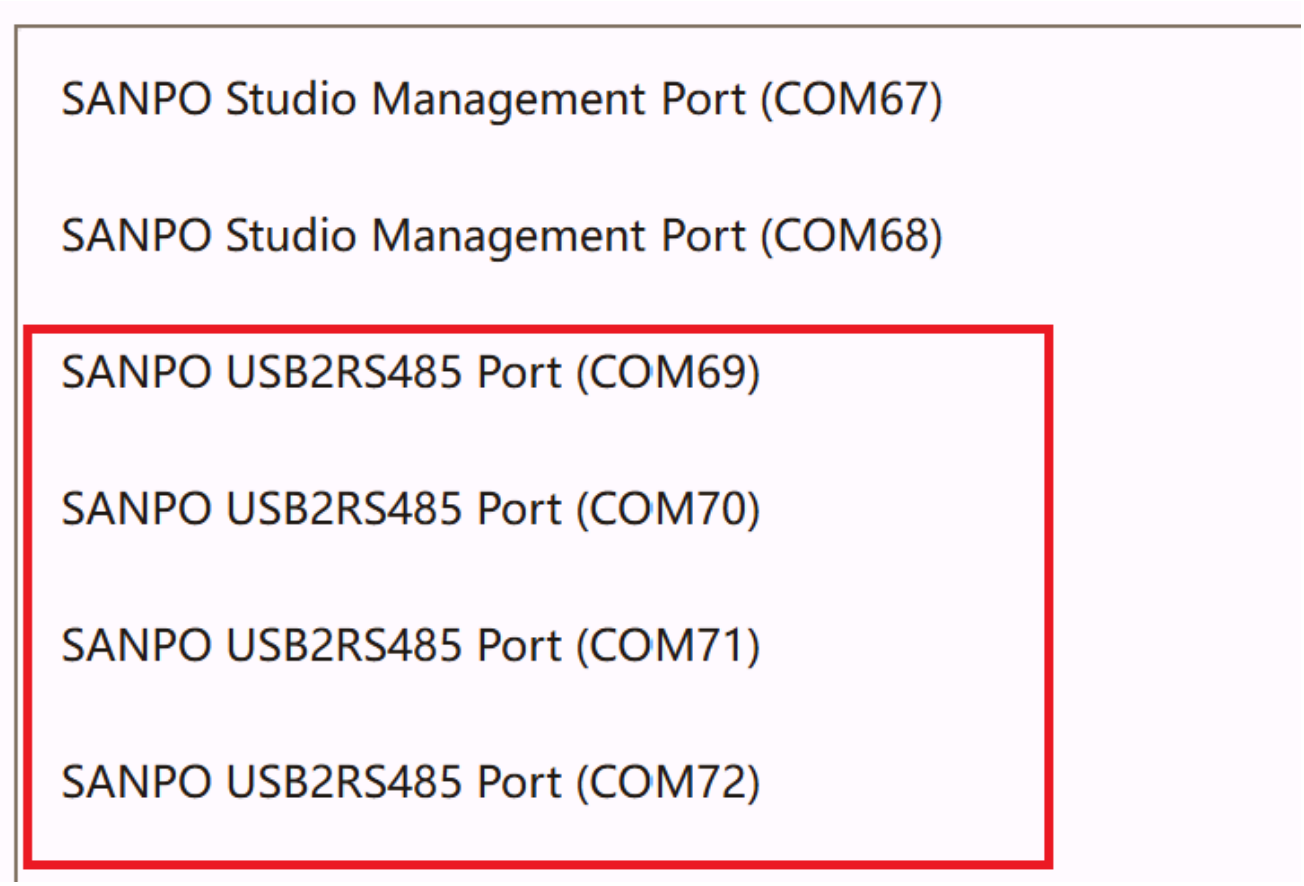
现象	处理方法
接入USB为什么会显示6个串口	6个串口中，2个管理串口用于USB协议模式，4个RS485串口用于RS485透传
连接管理串口，下发指令无反馈	尝试连接另外1个管理串口，2个串口分别对应CAN/RS485-1/2和CAN/RS485-3/4，但是枚举顺序与电路板上的丝印不会对应，第一次接入后，确定对应关系，下次接入，对应关系不会改变
SocketCAN枚举的CANX不对应	SocketCAN枚举的CAN0/CAN1/CAN2/CAN3与电路板上丝印不会对应，第一次接入后，确定对应关系，下次接入，对应关系不会改变

提示： Linux/Windows枚举串口和网络接口的顺序，量产时可做自动识别，出厂初始化自动识别对应关系，写入配置文件。

附录1： USB/SPI 接入说明，串口/网络的选择

USB 转 RS485 透传模式

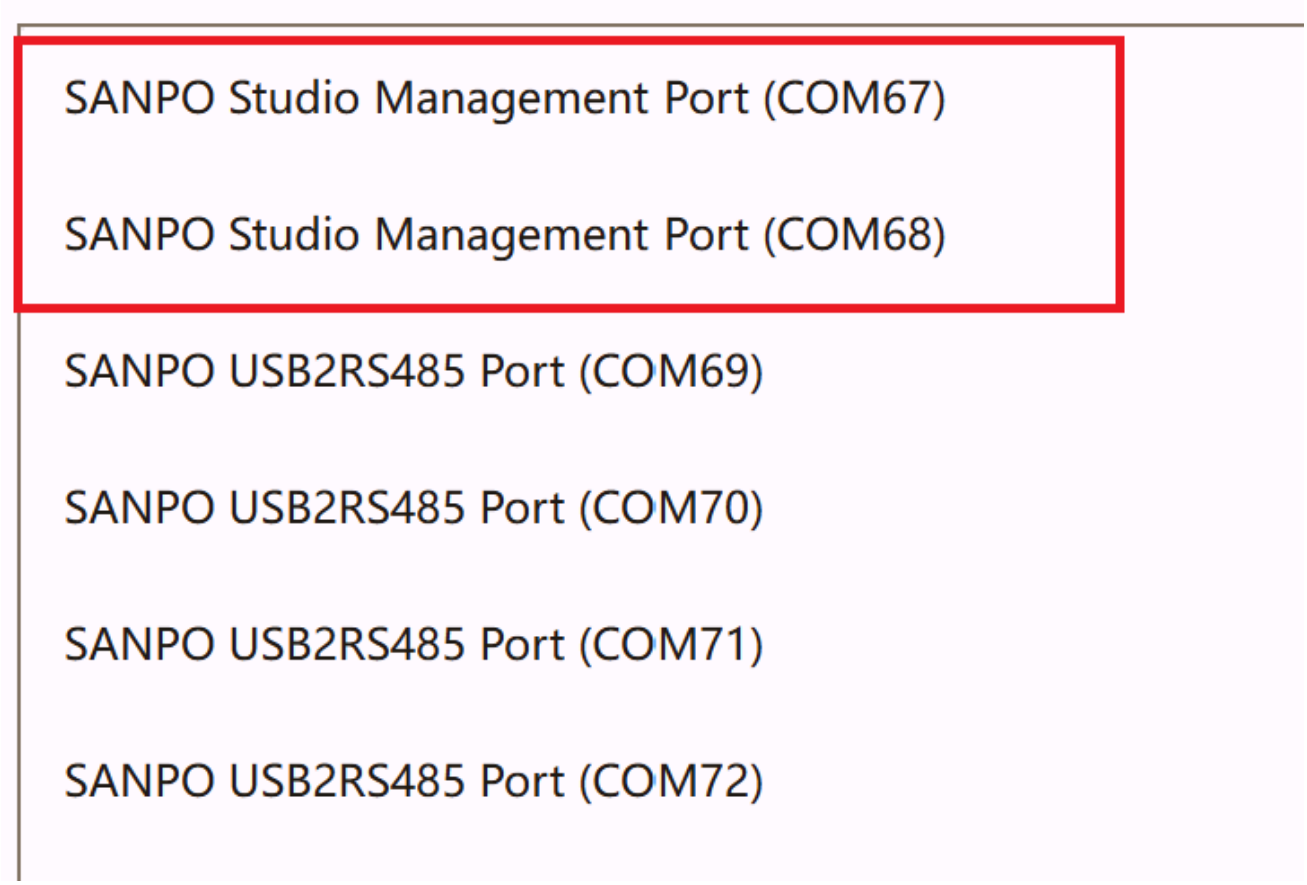
USB 接入电脑或者主控板后，会出现 4 个 `SANPO USB2RS485` 串行端口，每1个串口对应1路RS485通道，打开串口时使用的波特率等参数会同步给RS485总线，数据完全透传。



点击查看 Windows下如何识别RS485串口
点击查看 Linux 下如何识别RS485串口

USB 转 RS485 协议模式

在原始数据前后加上指定的报文头和报文尾，SANPO SPINE 收到数据后，去掉增加的报文头和尾，把原始数据再转发给 RS485 设备。协议模式使用 2个 `SANPO Studio Management Port` 串行端口，每1个串口对应1颗板载的STM32芯片，每颗芯片控制2路RS485通道。



点击查看 Wiindows下如何识别RS485串口
点击查看 Linux 下如何识别RS485串口

USB 转 FDCAN/CAN 透传模式

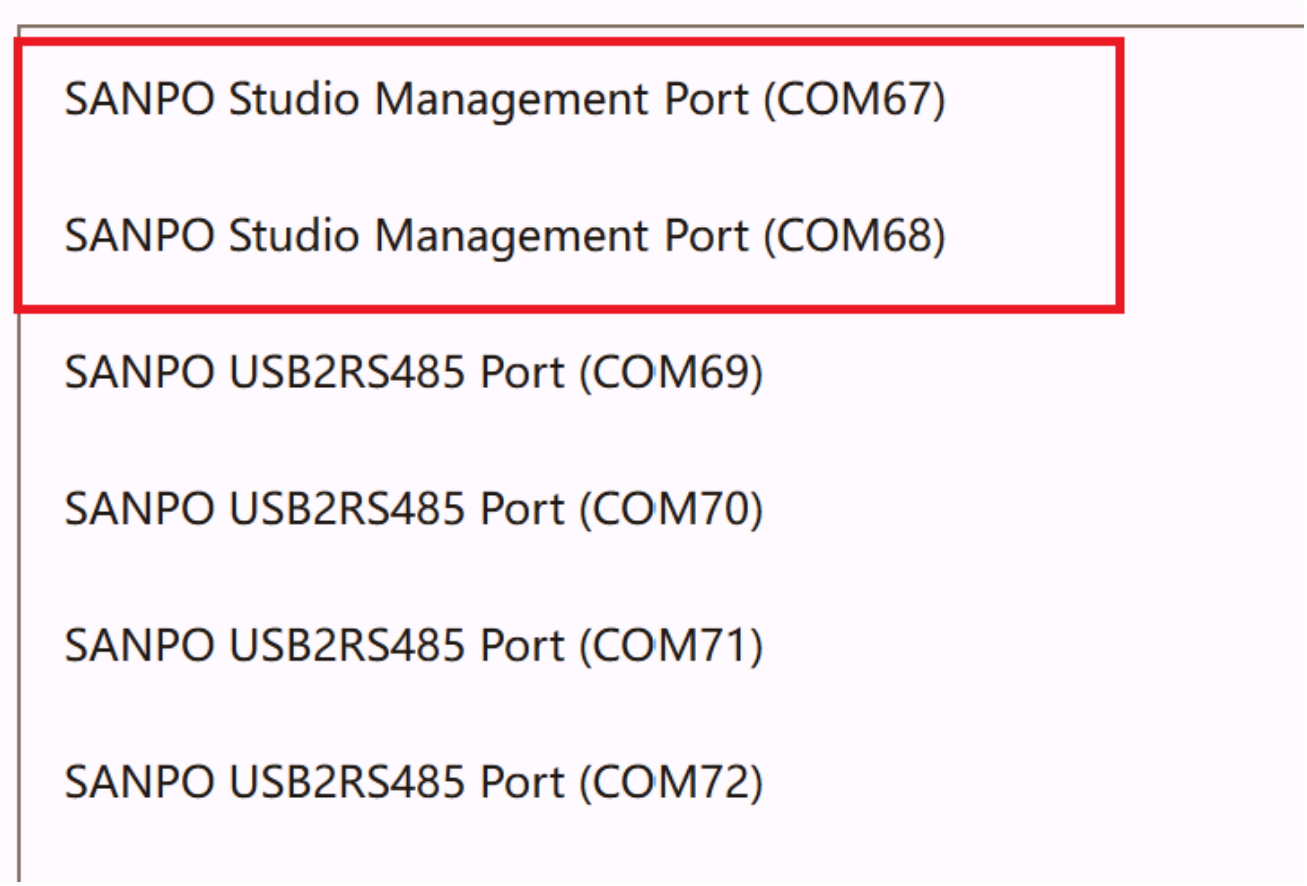
使用 Linxu SocketCAN，PythonCAN，candlelight等方式进行控制。例如下图是 Linux 下使用SocketCAN查看4路CAN通道(Linux 默认编号 CAN0/CAN1/CAN2/CAN3)

```
sanpo@sanpo-ubuntu:~$ ip link show type can
4: can0: <NOARP,UP,LOWER_UP,ECHO> mtu 72 qdisc pfifo_fast state UP mode DEFAULT group default qlen 10
    link/can
5: can1: <NOARP,UP,LOWER_UP,ECHO> mtu 72 qdisc pfifo_fast state UP mode DEFAULT group default qlen 10
    link/can
6: can2: <NOARP,UP,LOWER_UP,ECHO> mtu 72 qdisc pfifo_fast state UP mode DEFAULT group default qlen 10
    link/can
7: can3: <NOARP,UP,LOWER_UP,ECHO> mtu 72 qdisc pfifo_fast state UP mode DEFAULT group default qlen 10
    link/can
```

USB 转 FDCAN/CAN 协议模式

在原始数据前后加上指定的报文头和报文尾，SANPO SPINE 收到数据后，去掉增加的报文头和尾，把原始数据再转发给FDCAN/CAN设备。协议模式使用 2个 `SANPO Studio Management Port` 串行端口，每1个串口对应1颗板载的STM32芯片，每颗芯片控制2路CAN/FDCAN通

道。

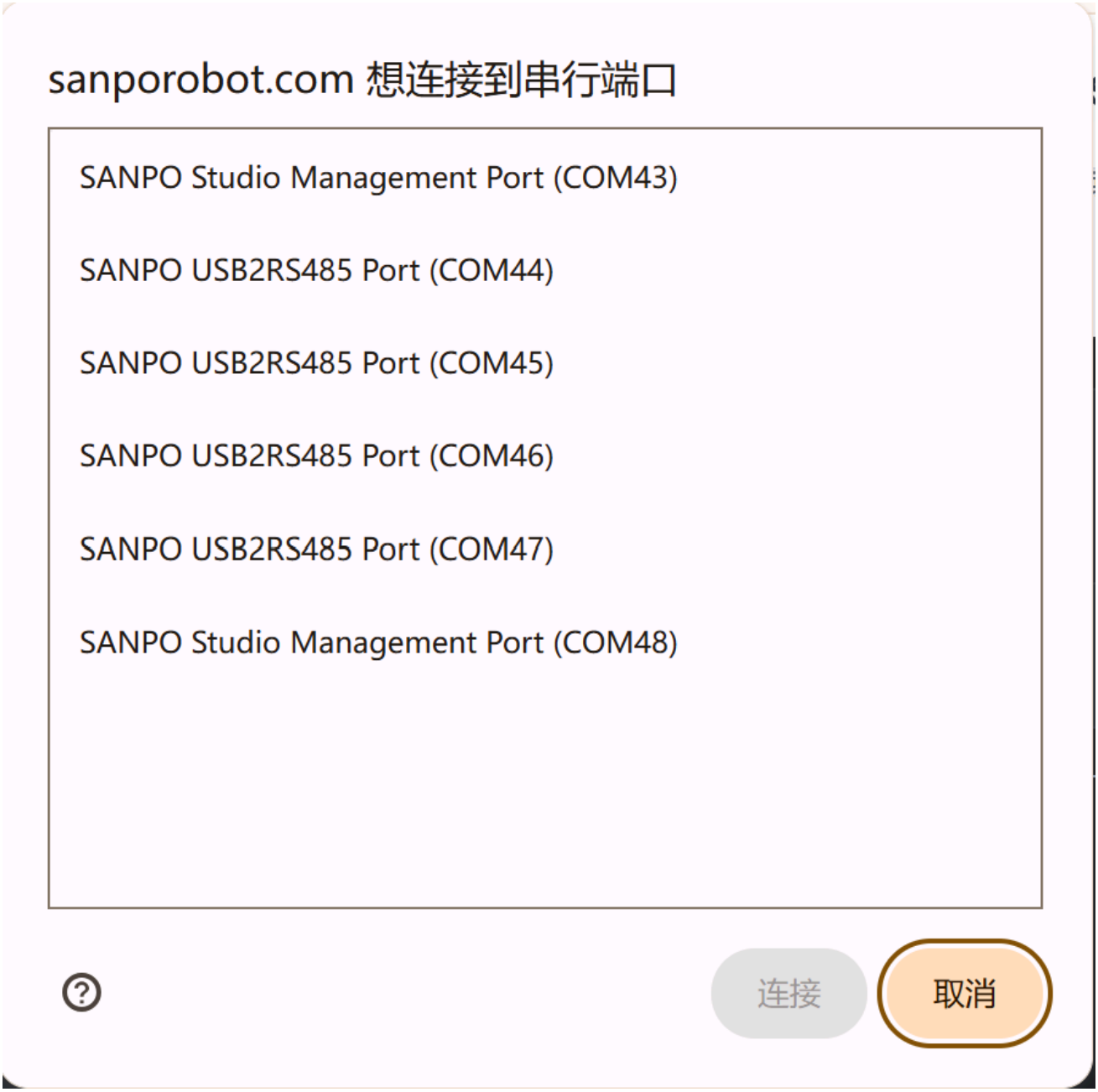


点击查看 Windows下如何识别RS485串口
点击查看 Linux 下如何识别RS485串口

附录2：Windows下 如何识别管理口和 RS485 透传口

在 Windows 中打开 SANPO Studio，点击“连接”，浏览器的串口选择窗口会直接显示端口名称：

- 使用 SANPO Studio 或协议模式时，选择 `SANPO Studio Management Port`；
- 使用 RS485 原始数据透传时，在自己的串口程序中选择 `SANPO USB2RS485 Port`；



SANPO Studio 的连接窗口可以区分端口类型，但不能自动判断某个端口对应整板上的 RS485-A/B/C/D。第一次使用仍应逐路连接或测试，并在自己的上位机中保存端口与物理接口的对应关系。

附录3：Linux下 如何识别管理口和 RS485 透传口

Linux 下可以按照串口识别工具使用说明运行 `identify_sanpo_serial_ports_v8.py`。脚本只读取 USB 串口信息，不会打开端口，也不会发送任何数据：

```
cd demo/rs485
python3 -m pip install pyserial
python3 identify_sanpo_serial_ports_v8.py
```

```
jetson@nano:~/demo_v8/rs485$ python3 identify_sanpo_serial_ports_v8.py
PORT          ROLE                          MCU SERIAL                IF   LOCATION
/dev/ttyACM3  SANPO Studio Management Port  203E35923442              1    1-2.1.2:1.1
/dev/ttyACM4  SANPO USB2RS485 Port 1        203E35923442              3    1-2.1.2:1.3
/dev/ttyACM5  SANPO USB2RS485 Port 2        203E35923442              5    1-2.1.2:1.5
/dev/ttyACM0  SANPO Studio Management Port  2057364A5731              1    1-2.1.1:1.1
/dev/ttyACM1  SANPO USB2RS485 Port 1        2057364A5731              3    1-2.1.1:1.3
/dev/ttyACM2  SANPO USB2RS485 Port 2        2057364A5731              5    1-2.1.1:1.5

Found 6 serial port(s) on 2 MCU(s): 2 management, 4 RS485, 0 unknown.
MCU serial numbers group ports but do not identify PCB A/B/C/D order.
```

输出中的 `MCU SERIAL` 用于把一个管理口和同一颗 MCU 的两个 RS485 透传口归为一组；`Port 1`、`Port 2` 只表示该 MCU 内部的两路 RS485。USB 序列号和 `/dev/ttyACM*` 编号都不能自动确定整板 A/B/C/D 顺序，首次接线后应记录实际映射。

需要程序读取结果时，可以输出 JSON：

```
python3 identify_sanpo_serial_ports_v8.py --json
```

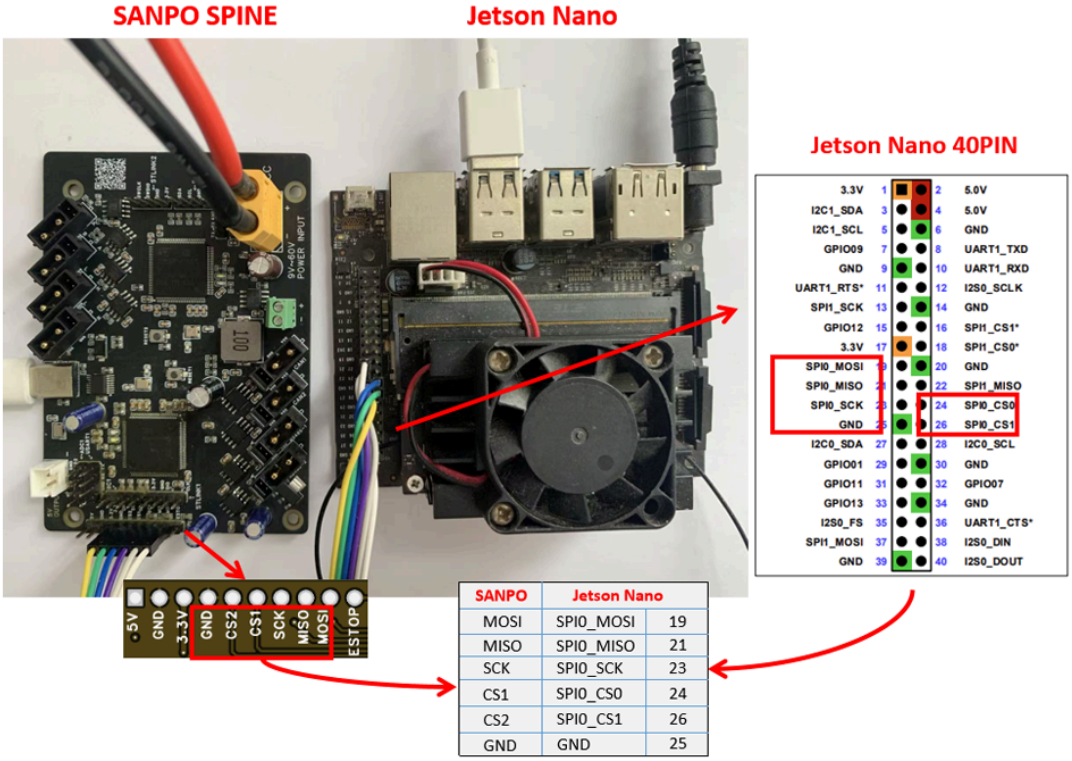
附录4：USB 协议模式下报文头的定义

协议模式下，使用2个管理串口用于 USB转CAN/FDCAN/RS485 通信，封装帧头定义如下(透传模式下无封装，直接传输原始数据)：

帧头	十六进制	用途
`ST`	`53 54`	经典 CAN 标准帧
`ET`	`45 54`	经典 CAN 扩展帧
`SF`	`53 46`	CAN FD 标准帧
`EF`	`45 46`	CAN FD 扩展帧
`RT`	`52 54`	RS485 封装帧

附录5：Jetson Nano SPI 接线示例

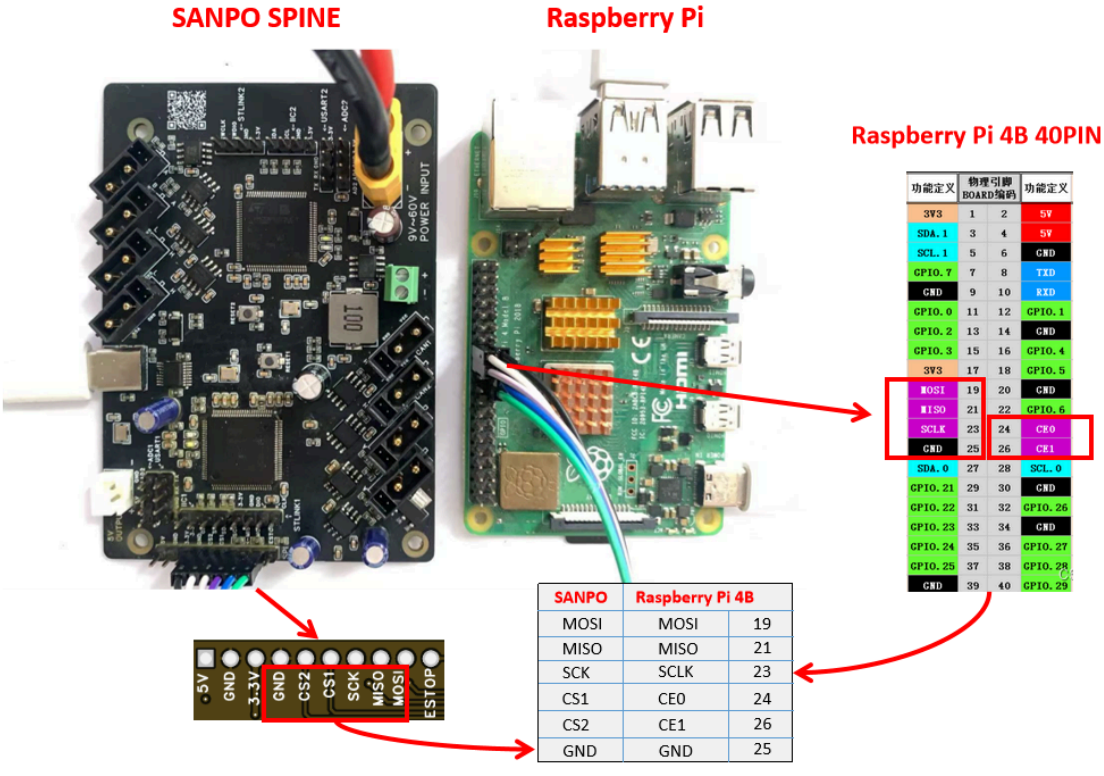
下图中的 `GND` 必须连接。即使改用 USB 给SANPO SPINE 集成板供电，也应保留 Jetson 与集成板 SPI 接口之间的 `GND` 连接。尽量连接2根 `GND` 到 Jetson。



SANPO	Jetson Nano	
MOSI	SPI0_MOSI	19
MISO	SPI0_MISO	21
SCK	SPI0_SCK	23
CS1	SPI0_CS0	24
CS2	SPI0_CS1	26
GND	GND	25

附录6： 树莓派 SPI 接线示例

下图中的 `GND` 必须连接。即使改用 USB 给SANPO SPINE 集成板供电，也应保留 Jetson 与集成板 SPI 接口之间的 `GND` 连接。尽量连接2根 `GND` 到 Jetson。



SANPO	Raspberry Pi 4B	
MOSI	MOSI	19
MISO	MISO	21
SCK	SCLK	23
CS1	CE0	24
CS2	CE1	26
GND	GND	25